ML CLUB
NIT SILCHAR

CORE TEAM TASK

Neural Network Implementation Challenge

A one-week, hands-on build: implement a neural network from scratch in NumPy, then rebuild it with a framework — and present what you actually understand.

DURATION
1 Week

PRESENTATION
Saturday, Offline Meet

FORMAT
Individual + Leaderboard

1. Goal of This Task

You already know the basics of ML — cost functions, gradient descent, what a model "learns." This task is about closing the gap between **knowing the theory** and **being able to implement it yourself.**

By the end of this week, you should be able to confidently answer, without hesitation:

- What is the shape of every matrix in your network, and why?
- What exactly happens in a forward pass, layer by layer?
- What exactly happens in a backward pass — where do the gradients come from?
- How does the network actually learn (what does "training" mean mechanically)?
- How is loss calculated, and what does the number mean?
- How do you know if your model is actually good (accuracy, confusion matrix)?

> **RULE #1 — NO AI-GENERATED CODE**
>
> You may use AI to ask conceptual questions ("what is a Jacobian," "explain chain rule again") but not to write or fix your implementation. You must be able to explain every line you submit — this will be tested live during your presentation via questions.

2. The Dataset — Same for Everyone

We're deliberately keeping the dataset simple and identical for everyone, so nobody spends time on data cleaning — the whole point this week is the network itself.

Use `sklearn.datasets.load_digits()` — 1,797 images of handwritten digits (0–9), each 8×8 pixels, flattened into 64 numbers. 10 output classes.

Use exactly this loading code so everyone works with the same split:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
X = X / 16.0  # normalize pixel values (originally 0-16) to 0-1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, y_train.shape)  # sanity check your shapes
```

Do not change the `random_state` or `test_size` — keeping this identical means everyone's results are actually comparable during presentations.

3. Part 1 — Build a Neural Network From Scratch

No `torch`, `tensorflow`, `keras`, `sklearn.neural_network`, or any autograd library for this part. Just NumPy. This is the core of the task.

A Design your architecture before writing code

Pick a simple architecture, e.g. `64 → 32 → 16 → 10` (input → hidden → hidden → output). You can choose your own hidden layer sizes, but keep it to 2–3 hidden layers max — no need to go deep.

Before writing any code, write out on paper (and keep in your slides for presentation):

- Shape of `X`, `W1`, `b1`, `Z1`, `A1`
- Shape of `W2`, `b2`, `Z2`, `A2` ... all the way to your output layer
- Which activation function you're using at each layer, and why (e.g. ReLU for hidden layers, softmax for output since this is multiclass classification)

This shape diagram is mandatory to bring to your presentation.

B Implement the forward pass

Write the following **yourself**, without importing them:

- Linear transformation at each layer (`Z = W·A_prev + b`)
- Activation functions: ReLU (hidden layers), Softmax (output layer)
- Cross-entropy loss function

After writing it, print the shape of every intermediate value and check it matches your Step A diagram exactly.

C Derive and implement the backward pass

This is the most important part of the week — give it the most time.

- On paper, derive the gradient of the loss with respect to each layer's `Z`, `W`, and `b`, working backward from the output.
- Implement this in code, layer by layer.
- Common bugs to watch for: wrong axis in `np.sum` when computing `db`, forgetting to transpose in matrix multiplications, mismatched shapes between forward and backward pass.

D Gradient checking

Before trusting your training loop, verify your analytical gradients against numerically estimated ones for a few weights:

```
grad_approx = (Loss(W + eps) - Loss(W - eps)) / (2 * eps)
```

If your analytical gradient and numerical gradient are close (relative difference < 1e-5 or so), your backward pass is correct. This will catch almost every backprop bug — use it.

E **Training loop**

- Implement mini-batch (or full-batch) gradient descent — plain SGD, no need for Adam/momentum.
- Track and plot the loss over epochs — you should see it decreasing.
- Train until your model reaches reasonable accuracy (should comfortably exceed 90% test accuracy with a small MLP — if you're far below that, something is likely wrong).

F **Evaluate**

- Compute test accuracy.
- Plot a confusion matrix.
- Look at 3–5 misclassified digits and briefly note anything you notice.

WHAT TO SKIP THIS WEEK

Regularization (L2, dropout) · Batch normalization · Learning rate schedules · Adam/momentum optimizers · Convolutional layers

4. Part 2 — Rebuild With a Framework

Once Part 1 works, rebuild the **same architecture** using PyTorch or TensorFlow/Keras (your choice). This should only take an hour or two.

- Use the framework's built-in layers, autograd, and optimizer (`nn.Linear`, `.backward()`, `optim.SGD`, etc.)
- Train it on the same dataset, same split.
- Compare against your Part 1 results: how many lines of code did you save? What exactly did the framework abstract away for you? Do you get similar accuracy and a similar loss curve?

Bring this comparison as a short slide or two — this is meant to make you appreciate what libraries like PyTorch handle for you *after* you've done it by hand, not before.

5. Deliverables Checklist

Bring all of this to Saturday's presentation:

- [ ] Shape diagram of your architecture (Step A)
- [ ] Working NumPy-only implementation (forward pass, loss, backward pass, training loop)
- [ ] Evidence of gradient checking (even just a printed comparison for a couple of weights)
- [ ] Loss curve plot
- [ ] Accuracy + confusion matrix on test set
- [ ] Framework (PyTorch/Keras) version of the same network
- [ ] Short comparison of scratch vs. framework version
- [ ] Code, in a state you can walk through live

6. Presentation Guidelines

Format: Offline meet, Saturday. 10–15 minutes per person — code walkthrough and/or slides, your choice, but must show actual working code and outputs (not just a slide describing it).

What to cover:

1. Your architecture and shape diagram
2. Walkthrough of your forward pass code
3. Walkthrough of your backward pass code (explain where the gradients come from — this is the part we'll press on with questions)
4. Your training loop and loss curve
5. Final accuracy / confusion matrix
6. What changed in the framework version, and what it abstracted away

Be ready for questions from batchmates and core members — explaining under questioning is part of what's being scored.

Scoring & Leaderboard

Batchmates will score each presenter on a 1–5 scale across these axes (not just one overall number):

Criteria	What it's assessing
Clarity of explanation	Could you actually follow their forward/backward pass logic?
Depth of understanding	Did they explain *why* (why this activation, why this loss), not just *what*?
Code correctness / quality	Does the shape logic make sense? Does it actually train and converge?

Handling questions	How well did they respond when questioned?

Final score will be a blend of peer scores and core team scores. Leaderboard will be shared after all presentations.

7. Haven't Learned Neural Networks Yet?

If your batch hasn't formally covered neural networks yet, don't panic — a week is enough to get functional if you focus. Two options, and doing both is ideal.

Option A — Ask a Batchmate or Core Member

Genuinely the fastest way. Someone in your batch has already done deep learning — ask them to walk you through forward pass, backward pass, and gradient descent for a simple 2-layer network on a whiteboard. 30–45 minutes with a peer will get you further than hours of solo video-watching. Core members are also happy to explain — just ask.

Option B — Fast-Track Resources (~1–2 focused days)

1. **3Blue1Brown's Neural Networks series** (YouTube, 4 videos, ~1 hour total) — the best visual intuition for what forward pass, backprop, and gradient descent actually mean geometrically. Watch this first.
2. **Andrew Ng's Deep Learning Specialization, Course 1** (Neural Networks and Deep Learning) — same teaching style, covers forward/backward propagation and vectorized implementation. You only need the first 3–4 weeks of Course 1, not the whole specialization.
3. **CS231n (Stanford) notes on backpropagation** — a short written derivation of backprop with matrix shapes explicitly worked out; useful as a reference while deriving your own gradients in Step C.

Search for these titles directly. Recommend watching 3Blue1Brown first for intuition, then skimming the relevant Andrew Ng lectures while you're actually implementing — not finishing the whole course passively before starting to code.

> **DON'T**
>
> Try to "finish a course" before starting the task. Watch/read just enough to understand forward pass and backprop conceptually (half a day, max), then start implementing — you'll learn the rest by actually building it and getting stuck.

8. Quick FAQ

Can I use a different architecture than 64→32→16→10?

Yes, as long as it's a reasonable MLP (2–3 hidden layers). Just be ready to justify your choice.

What if my gradient check doesn't pass?

That means there's a bug in your backward pass — don't skip this and move to training with broken gradients. Debug it; this is the most valuable checkpoint in the whole task.

What if I can't hit 90% accuracy?

First check your gradient checking passed. If it did, check learning rate (too high/low is the most common issue), number of epochs, and that your loss is actually decreasing. Ask a core member if you're stuck for more than an hour.

Is PyTorch or Keras preferred for Part 2?

Either is fine — use whichever you're more comfortable with, or use this as a chance to try PyTorch if you haven't yet.

Good luck — see everyone Saturday.